UART Communication System

Simulation, Synthesis, and Timing Report

Omar Sameh Wadea Ali Mohamed
Mohamed Ahmed Abdelrahman
Yehia Mohamed Ahmed Elkharashy
Youssef Shady Gabr Abdo

Overview

This report details the simulation, elaboration, physical implementation, and static timing analysis of a Universal Asynchronous Receiver-Transmitter (UART) system. The design consists of a transmitter (`uart_tx`) and a receiver (`uart_rx`) operating at a defined baud rate and clock frequency.

UART Protocol Fundamentals

The Universal Asynchronous Receiver-Transmitter (UART) is a standard hardware protocol used for serial communication between digital devices. Unlike synchronous protocols (such as SPI or I2C), UART is asynchronous, meaning it does not use a shared clock wire to synchronize the sender and receiver. Instead, it relies on just two data wires: Transmit (TX) and Receive (RX).

To communicate successfully without a shared clock, both devices must be configured to the exact same communication speed, known as the **Baud Rate**. Data is transmitted one bit at a time using a specific frame structure:

- **Start Bit:** The transmitting device pulls the idle line from a high state (1) to a low state (0), signaling the receiver to start its internal timer.
- **Data Bits:** The actual data payload (typically 8 bits) is shifted out sequentially, allowing the receiver to sample the line at the center of each bit period.
- **Stop Bit:** The line is pulled back to a high state (1) to indicate the end of the data packet and return the line to its idle state.

Verilog RTL Implementation

The provided Verilog codebase models the UART protocol using parameterized structural logic and Finite State Machines (FSMs). The architecture is divided into three main operational mechanics:

Baud Rate Generation

Since UART is asynchronous, timing is managed internally by dividing the system clock (`CLK_FREQ`, 50 MHz) by the target transmission speed (`BAUD_RATE`, 115200). This mathematical ratio defines the `BIT_PERIOD`. A dedicated counter (`clk_cnt`) increments on every clock cycle to track when to shift or sample the next bit on the line.

Transmitter (`uart_tx`)

The transmitter is driven by a 4-state FSM:

- **IDLE:** The `tx_out` line is held high. When the `start` signal is asserted, the input `tx_data` is latched into an internal register, and the FSM transitions to START.
- **START:** The `tx_out` line is pulled low for one full `BIT_PERIOD`.
- **DATA:** The FSM holds for 8 bit periods, using a `bit_index` counter to sequentially shift out the latched data to the `tx_out` line.
- **STOP:** The `tx_out` line is pulled high again for one bit period to signal the end of the frame, pulling the `busy` flag low before returning to IDLE.

Receiver (`uart_rx`)

The receiver utilizes a similar 4-state FSM but includes logic for mid-bit sampling to ensure data stability and avoid noise glitches:

- **IDLE:** The module continuously polls `rx_in` for a high-to-low transition (the Start Bit).
- **START:** The counter waits for exactly half of a bit period (`BIT_PERIOD / 2`). If the line remains low at this midpoint, it confirms a valid start bit and proceeds to DATA.
- **DATA:** The FSM waits a full bit period between samples, grabbing the incoming bit precisely at the center of its duration (where the signal is most stable) and shifting it into an internal `shift_reg`.
- **STOP:** After 8 bits are received, it waits for the stop bit period, outputs the assembled byte to `rx_data`, and raises the `rx_done` flag.

1. Behavioral Simulation (QuestaSim)

The behavioral simulation verifies the logical correctness of the UART protocol. The automated testbench successfully compares the transmitted byte against the received byte, printing a success message to the console upon completion. The waveform demonstrates the state machine transitions (IDLE, START, DATA, STOP) functioning exactly as intended while successfully shifting out and reconstructing the `8'hA5` data byte.

```
VSIM 8> run -all
# SUCCESS: Sent 8'hA5 and received 8'hA5 correctly!
# ** Note: $finish    : tb_uart.v(63)
#    Time: 82830 ns  Iteration: 0  Instance: /tb_uart
# 1
# Break in Module tb_uart at tb_uart.v line 63
```

Figure 1: QuestaSim Console: Successful verification of transmitted vs. received data

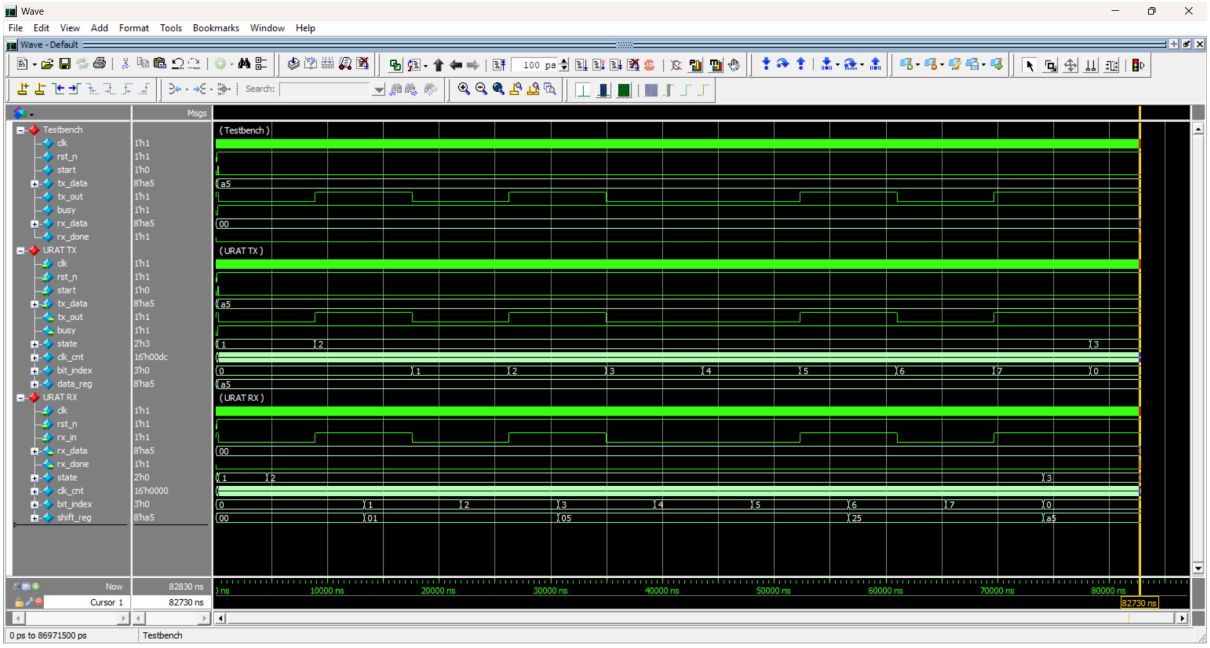

Figure 2: QuestaSim Waveform: Full UART Transaction of 8'hA5

2. RTL Elaboration (Vivado)

The Elaborated Design translates the Verilog code into generic structural logic before target-specific optimization.

Top-Level Architecture

The top-level schematic illustrates the direct interconnection between the transmitter (`u_tx`) and the receiver (`u_rx`) via the serial `tx_out` to `rx_in` line.

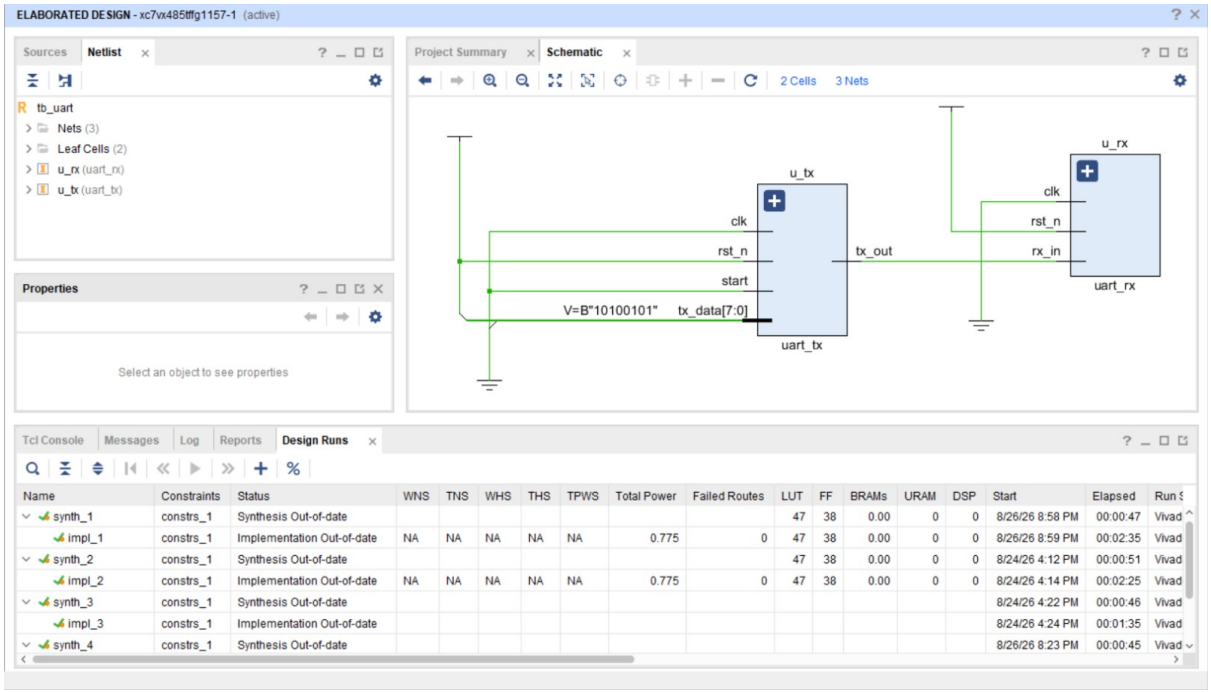

Figure 3: Elaborated Design: Top-Level Block Diagram

Internal Module Schematics

The following schematics display the internal elaborated logic, including multiplexers, comparators, and registers used to track the baud rate counters and shift registers.

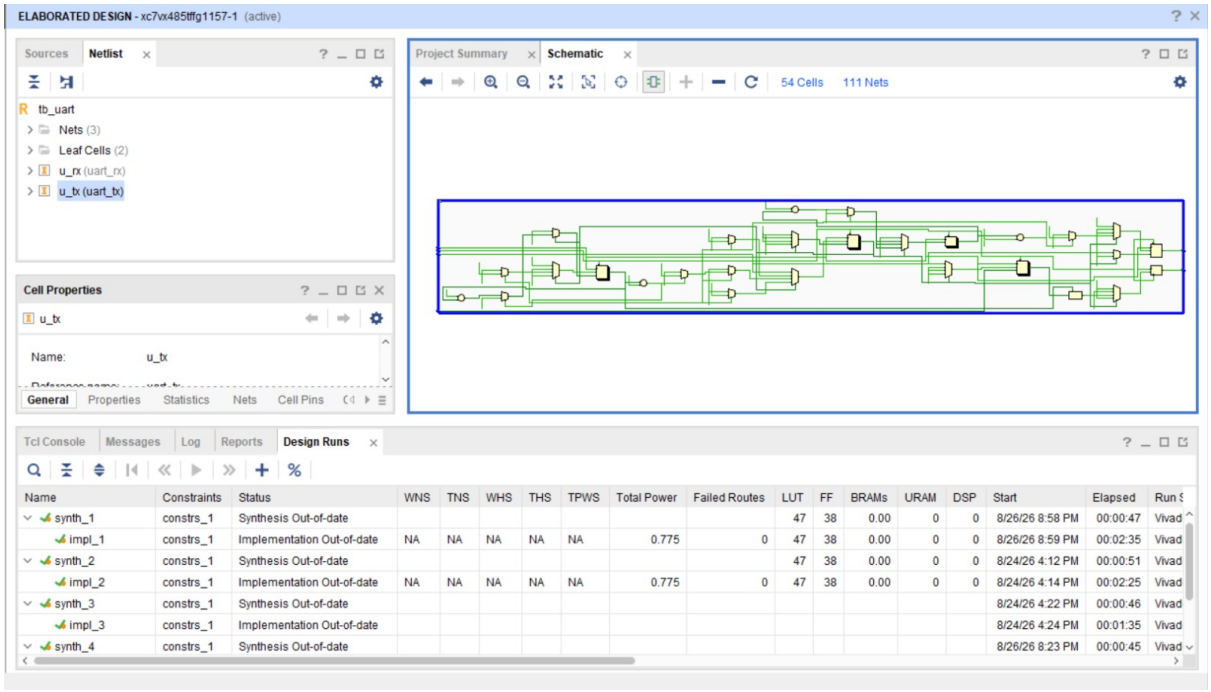

Figure 4: Elaborated Design: Transmitter (`u_tx`) Internal Logic

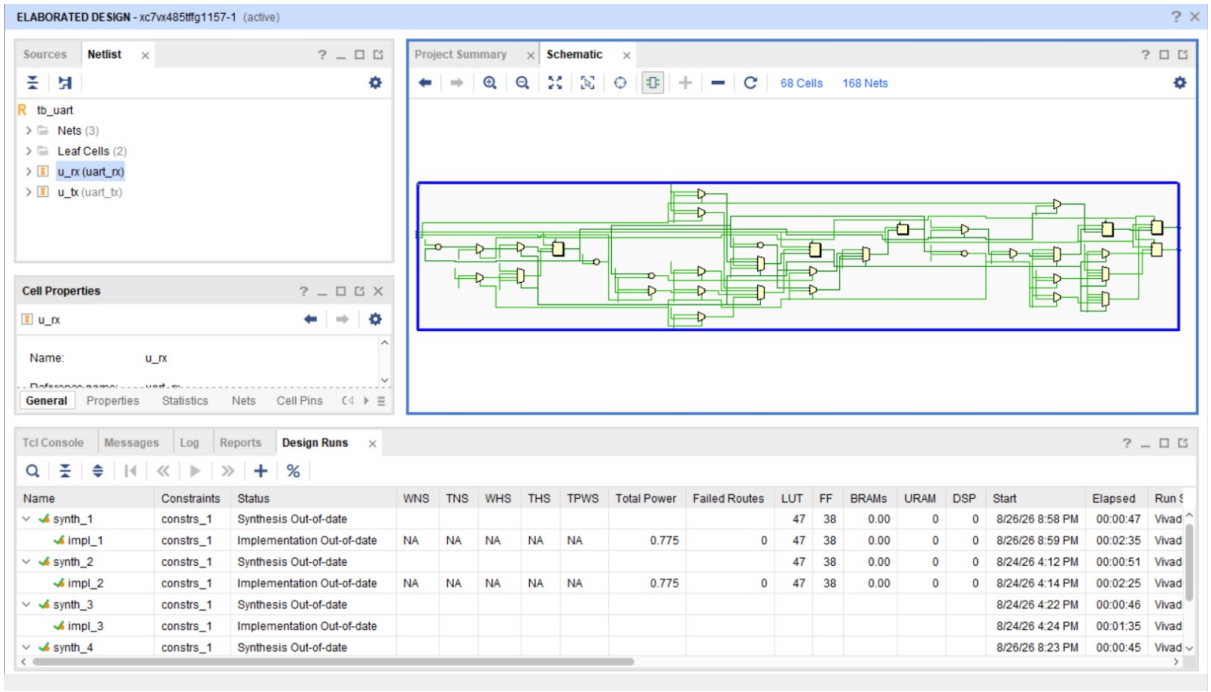

Figure 5: Elaborated Design: Receiver (`u_rx`) Internal Logic

3. Synthesis and FSM Extraction

During synthesis, Vivado maps the generic RTL into device-specific primitives (LUTs and Flip-Flops). The synthesizer also successfully identifies and extracts the finite state machines (FSMs) for both the transmitter and receiver.

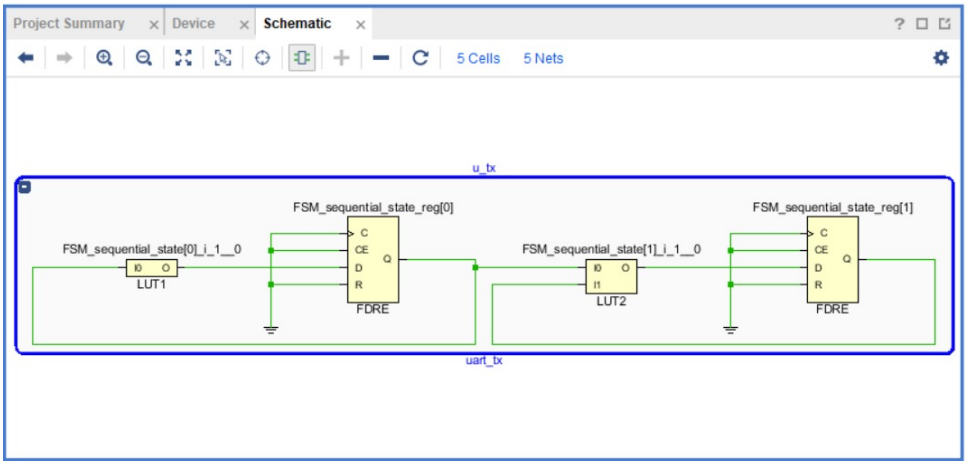

Figure 6: Synthesized Design: Transmitter FSM State Registers

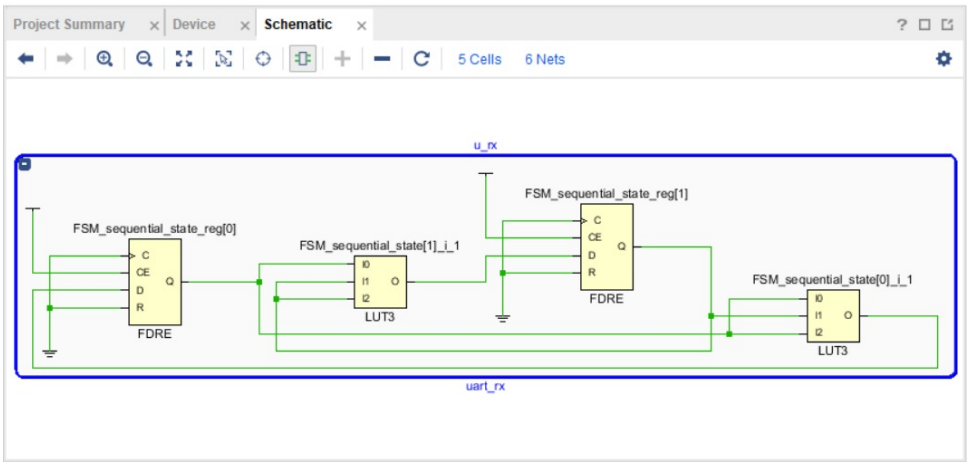

Figure 7: Synthesized Design: Receiver FSM State Registers

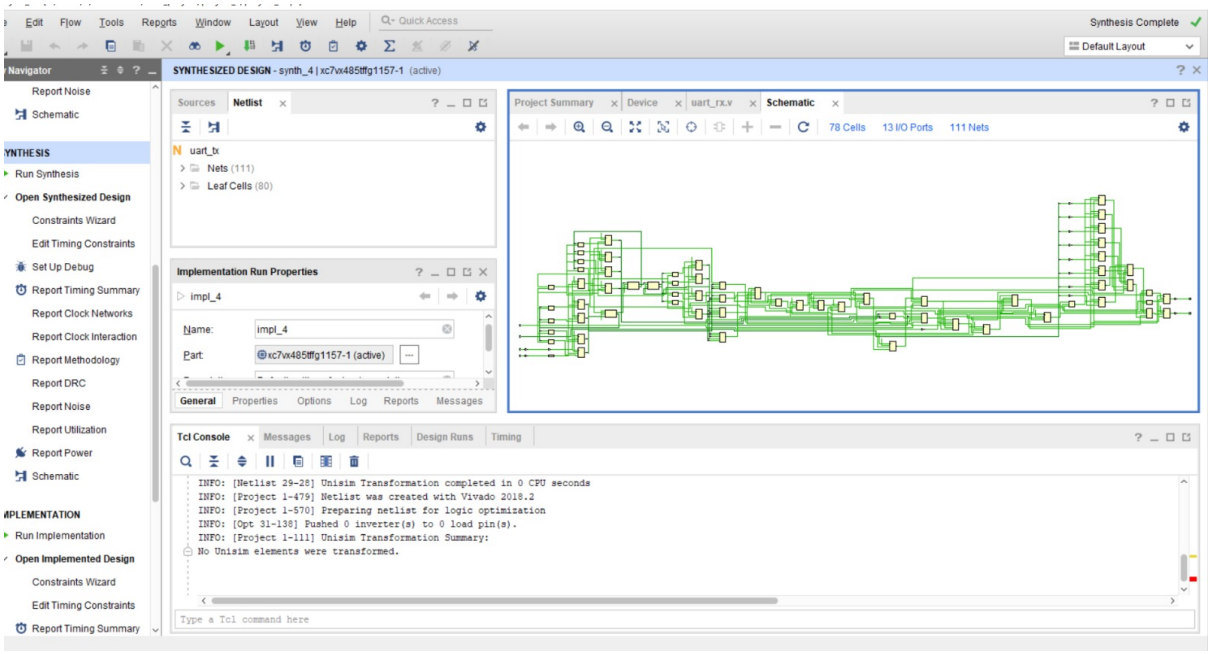

Figure 8: Synthesized Design: Full Flattened Netlist Schematic

4. Physical Implementation

The implemented design shows the final placement and routing of the synthesized netlist onto the actual FPGA silicon fabric. The resource utilization (LUTs and FFs) is highly efficient, taking up a minimal footprint on the device.

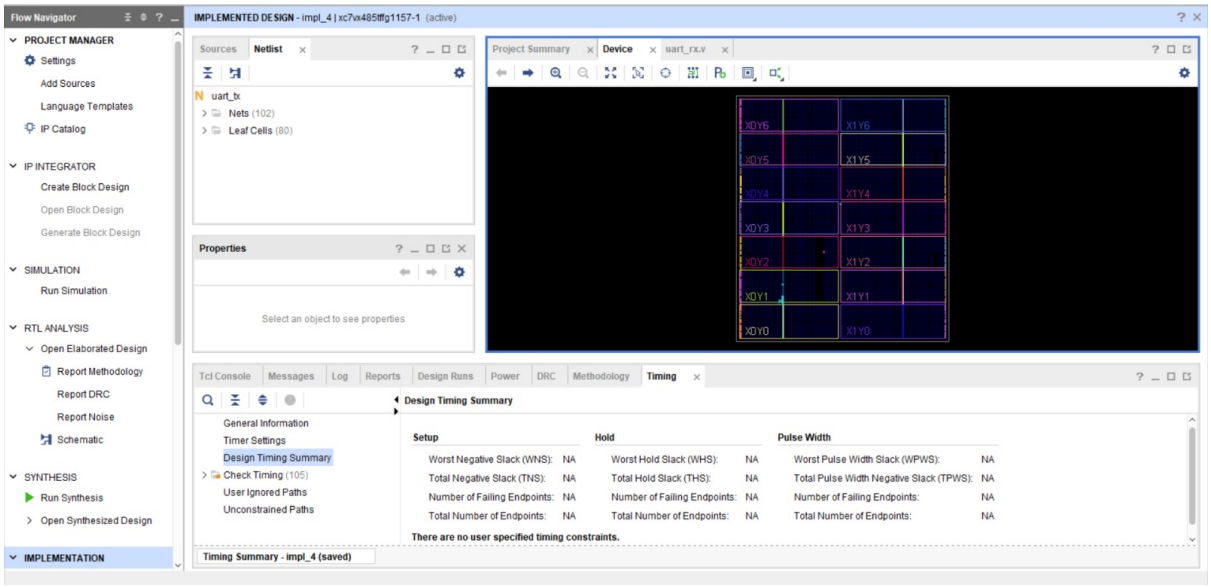

Figure 9: Implemented Design: FPGA Device Floorplan and Routing

5. Static Timing Analysis (STA)

Post-implementation timing analysis confirms that the design easily meets all timing constraints at the target clock frequency. There are zero failing endpoints, with a healthy Worst Negative Slack (WNS) for setup time and positive Worst Hold Slack (WHS) for hold time, ensuring reliable performance on the physical silicon without metastability issues.

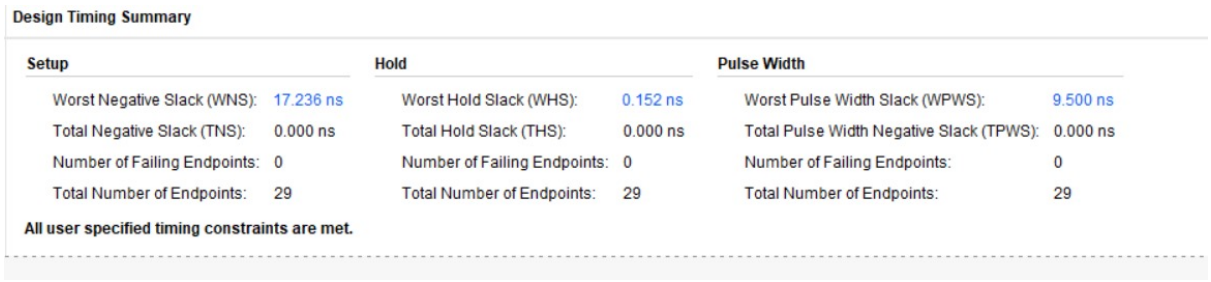

Figure 10: STA: Setup, Hold, and Pulse Width constraints met